arXiv:2408.10920v1 [cs.LG] 20 Aug 2024

Recurrent Neural Networks Learn to Store and Generate Sequences using Non-Linear Representations

Róbert Csordás
Stanford University
rcsordas@stanford.edu

Christopher Potts
Stanford University
cgpotts@stanford.edu

Christopher D. Manning
Stanford University
manning@stanford.edu

Atticus Geiger
Pr(Ai)^{2}R Group
atticusg@gmail.com

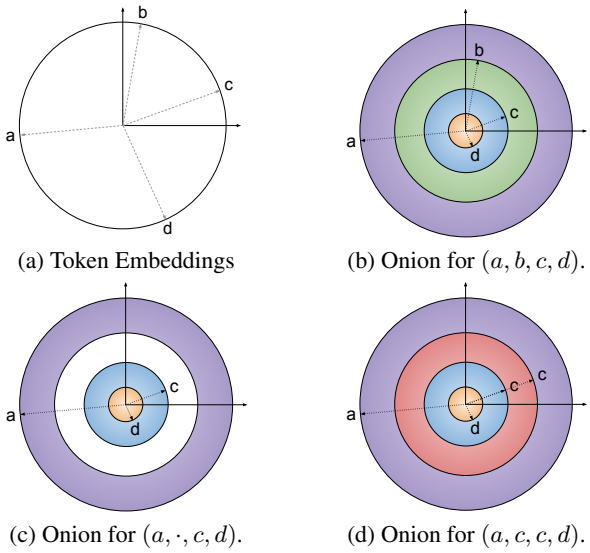
Abstract

The Linear Representation Hypothesis (LRH) states that neural networks learn to encode concepts as directions in activation space, and a strong version of the LRH states that models learn *only* such encodings. In this paper, we present a counterexample to this strong LRH: when trained to repeat an input token sequence, gated recurrent neural networks (RNNs) learn to represent the token at each position with a particular order of magnitude, rather than a direction. These representations have layered features that are impossible to locate in distinct linear subspaces. To show this, we train interventions to predict and manipulate tokens by learning the scaling factor corresponding to each sequence position. These interventions indicate that the smallest RNNs find only this magnitude-based solution, while larger RNNs have linear representations. These findings strongly indicate that interpretability research should not be confined by the LRH. [1]

1 Introduction

It has long been observed that neural networks encode concepts as linear directions in their representations (Smolensky, 1986), and much recent work has articulated and explored this insight as the Linear Representation Hypothesis (LRH; Elhage et al. 2022; Park et al. 2023; Guerner et al. 2023; Nanda et al. 2023; Olah 2024). A *strong* interpretation of the LRH says that such linear encodings are entirely sufficient for a mechanistic analysis of a deep learning model (Smith, 2024).

In this paper, we present a counterexample to the Strong LRH by showing that recurrent neural networks with Gated Recurrent Units (GRUs; Cho et al. 2014) learn to represent the token at each position using magnitude rather than direction when solving a simple repeat task (memorizing and generating a sequence of tokens). This leads to a set of layered features that are impossible to locate in distinct linear subspaces. We refer to the resulting hidden states as 'onion representations' to evoke how sequence position can be identified by iteratively peeling off these magnitude changes from the positions before it (Figure 1). In our experiments, this is the only solution found by the smallest networks (hidden size 48, 64); the larger networks (128, 512, 1024) learn to store input tokens in position-specific linear subspaces, consistent with the LRH, though we find these linear representations are compatible with onion-based mechanisms as well.



(a) Token Embeddings (b) Onion for (a, b, c, d). (c) Onion for $(a, \cdot, c, d)$. (d) Onion for (a, c, c, d).

Figure 1: We find that GRUs solve a repeat task by learning a scaling factor corresponding to each sequence position, leading to layered onion-like representations. In this simplified illustration, the learned token embeddings (a) are rescaled to have magnitudes proportional to their sequence positions (b). To change an element of the sequence, remove (c) and replace (d) the token embedding at the given positional magnitude. The layered nature of the representations makes them non-linear; any direction will cross-cut multiple layers of the onion.

We made this surprising finding in a hypothesis-driven fashion. Our Hypothesis 1 was that GRUs would store each token in a linear subspace. To

[1] Our code is public: https://github.com/robertcsordas/onion_representations

test this hypothesis, we employed a variant of distributed alignment search (DAS; Geiger et al. 2024b; Wu et al. 2023) that uses a Gumbel softmax to select dimensions for intervention. This revealed that the larger GRUs do in fact have linear subspaces for each position, but we found no evidence of this for the smaller ones (section 5). This led to Hypothesis 2: GRUs learn to represent input *bigrams* in linear subspaces. A DAS-based analysis supports this for the medium-sized models but not for the smallest ones (section 6). This left the task success of the smallest models to be explained.

For the smallest models, we observed that the update gates of the GRUs got gradually lower as the sequence progressed. This led to Hypothesis 3: onion representations. To evaluate this hypothesis, we learned interventions on the hidden vector encoding a sequence of tokens that replaces token A with token B at position j. The intervention adds the scaled difference of learned embeddings for A and B, where the scaling factor is determined by the position j with learned linear and exponential terms. Across positions, this intervention works with $\approx$90% accuracy, demonstrating the existence of layered features stored at different scales.

The existence of non-linear representations is a well-formed theoretical possibility. For example, under the framework of Geiger et al. (2024a) and Huang et al. (2024), any bijective function can be used to featurize a hidden vector, and interventions can be performed on these potentially non-linear features. However, the typical causal analysis of a neural networks involves only interventions on linear representations (see Section 2 for a brief review of such methods). We hope that our counterexample to the strong version of the LRH spurs researchers to consider methods that fall outside of this class, so that we do not overlook concepts and mechanisms that our models have learned.

2 Related Work

The Linear Representation Hypothesis Much early work on 'word vectors' was guided by the idea that linear operations on vectors could identify meaningful structure (Mikolov et al., 2013; Arora et al., 2016; Levy and Goldberg, 2014). More recently, Elhage et al. (2022) articulated the Linear Representation Hypothesis (LRH), which says that (1) features are represented as directions in vector space and (2) features are one-dimensional (see also Elhage et al. 2022; Park et al. 2023; Guerner et al. 2023; Nanda et al. 2023). Engels et al. 2024 challenged (2) by showing some features are irreducibly multi-dimensional. Olah (2024) subsequently argued that (1) is the more significant aspect of the hypothesis, and it is the one that we focus on here. Smith (2024) adds important nuance to the LRH by distinguishing a weak version (some concepts are linearly encoded) from a strong one (all concepts are linearly encoded).

Our concern is with the strong form; there is ample evidence that linear encoding is possible, but our example shows that other encodings are possible. In onion representations, multiple concepts can be represented in a linear subspace by storing each concept at a different order of magnitude, i.e., a 'layer' of the onion, and any direction will cross-cut multiple layers of the onion.

Intervention-based Methods Recent years have seen an outpouring of new methods in which interventions are performed on linear representations, e.g., entire vectors (Vig et al., 2020; Geiger et al., 2020; Finlayson et al., 2021; Wang et al., 2023), individual dimensions of weights (Csordás et al., 2021) and hidden vectors (Giulianelli et al., 2018; De Cao et al., 2020; Davies et al., 2023), linear subspaces (Ravfogel et al., 2020; Geiger et al., 2024b; Belrose et al., 2023), or linear features from a sparse dictionary (Marks et al., 2024; Makelov et al., 2024). These methods have provided deep insights into how neural networks operate. However, the vast and varied space of non-linear representations is woefully underexplored in a causal setting.

RNNs Recurrent Neural Networks (RNNs) were among the first neural architectures used to process sequential data (Elman, 1990, 1991). Many variants arose to help networks successfully store and manage information across long sequences, including LSTMs (Hochreiter and Schmidhuber, 1997) and GRUs (Cho et al., 2014). Bidirectional LSTMs provided the basis for one of the first large-scale pretraining efforts (ELMo; Peters et al. 2018). With the rise of Transformer-based models (Vaswani et al., 2017), RNNs fell out of favor somewhat, but the arrival of structured state-space models (Gu et al., 2021b,a; Gu and Dao, 2023; Dao and Gu, 2024) has brought RNNs back into the spotlight, since such models seek to replace the Transformer's potentially costly attention mechanisms with recurrent connections. We chose GRUs for our studies, with an eye towards better understanding structured state space models as well.

	$N = 48$	$N = 64$	$N = 128$	$N = 256$	$N = 512$	$N = 1024$
Exact-Match Accuracy	0.95 ± 0.01	0.97 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00

Table 1: Exact-match accuracy (mean of 5 runs; $\pm$ 1 s.d.) for GRUs of different sizes trained on the repeat task.

3 Models

In this paper, we focus on how RNNs solve the repeat task. As noted in section 2, this question has taken on renewed importance with the development of structured state-space models that depend on recurrent computations and are meant to provide efficient alternatives to transformers.

Define an RNN as $\boldsymbol{h}_t = f(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)$, $\boldsymbol{h}_0 = 0$, where $f(\cdot, \cdot)$ is the state update function, $t \in \{1, \ldots, T\}$ is the current timestep, $\boldsymbol{x}_t \in \mathbb{R}^N$ is the current input, and $\boldsymbol{h}_t \in \mathbb{R}^N$ is the state after receiving the input $\boldsymbol{x}_t$. The output of the model is $\boldsymbol{y}_t = g(\boldsymbol{h}_t)$. Vectorized inputs $\boldsymbol{x}_t$ are obtained with a learned embedding $\boldsymbol{E} \in \mathbb{R}^{N_S \times N}$, using the indexing operator $\boldsymbol{x}_t = \boldsymbol{E}[i_t]$, where $i_t \in \{1, \ldots, N_S\}$ is the index of the token at timestep t.

In our experiments, we use GRU cells over the more widely-used LSTM cells because they have a single state to intervene on, as opposed to the two states of the LSTM. GRU-based RNNs defined as:

$$\boldsymbol{z}_t = \sigma\left(\boldsymbol{W}_z \boldsymbol{x}_t + \boldsymbol{U}_z \boldsymbol{h}_t + \boldsymbol{b}_z\right) \quad (1)$$

$$\boldsymbol{r}_t = \sigma\left(\boldsymbol{W}_r \boldsymbol{x}_t + \boldsymbol{U}_r \boldsymbol{h}_t + \boldsymbol{b}_r\right) \quad (2)$$

$$\boldsymbol{u}_t = \tanh\left(\boldsymbol{W}_h \boldsymbol{x}_t + \boldsymbol{U}_h (\boldsymbol{r}_t \odot \boldsymbol{h}_t) + \boldsymbol{b}_h\right) \quad (3)$$

$$\boldsymbol{h}_t = (1 - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \odot \boldsymbol{u}_t \quad (4)$$

For output generation, we use $g(\boldsymbol{h}_t) = \mathrm{softmax}(\boldsymbol{h}_t \boldsymbol{W}_o + \boldsymbol{b}_o)$. The learned parameters are weights $\boldsymbol{W}_*, \boldsymbol{U}_* \in \mathbb{R}^{N \times N}$, and biases $\boldsymbol{b}_* \in \mathbb{R}^N$.

We will investigate how the final hidden state $\boldsymbol{h}_L$ of a GRU represents an input token sequence $\mathbf{i} = i_1, i_2, \ldots i_L$. The final state is a bottle-neck between the input token sequence and the output.

4 Repeat Task Experiments

Our over-arching research question is how different models learn to represent abstract concepts. The repeat task is an appealingly simple setting in which to explore this question. In this task, the network is presented with a sequence of random tokens $\mathbf{i} = i_1, i_2, \ldots, i_L$, where each i_j is chosen with replacement from a set of symbols N_S and the length L is chosen at random from $\{1 \ldots L_{\text{max}}\}$. This is followed by a special token, $i_{L+1} =$ 'S', that indicates the start of the repeat phase. The task is to repeat the input sequence: $y_{L+1+j} = i_j$. The variables in this task will represent positions in the sequence and take on token values.

As a preliminary step, we evaluate RNN models on the repeat task. The core finding is that all of the models solve the task. This sets us up to explore our core interpretability hypotheses in sections 5–7.

4.1 Setup

For our experiments, we generate 1M random sequences of the repeat task. The maximum sequence length is $L_{\text{max}} = 9$, and the number of possible symbols is $N_S = 30$. For testing, we generate an additional 5K examples using the same procedure, ensuring that they are disjoint at the sequence level from those included in the train set.

We use the same model weights during both the input and decoding phases. During the input phase, we ignore the model's outputs. No loss is applied to these positions. We use an autoregressive decoding phase: the model receives its previous output as input in the next step. We investigate multiple hidden state sizes, from $N = 48$ to $N = 1024$.

We train using a batch size of 256, up to 40K iterations, which is sufficient for each model variants to converge. We use an AdamW optimizer with a learning rate of 10^{-3} and a weight decay of 0.1.

4.2 Results

Table 1 reports on model performance at solving the repeat task. It seems fair to say that all the models solve the task; only the smallest model comes in shy of a perfect score, but it is at 95%. Overall, these results provide a solid basis for asking *how* the models manage to do this. This is the question we take up for the remainder of the paper.

5 Hypothesis 1: Unigram Variables

Intuitively, to solve the repeat task, the token at each position will have a different feature in the state vector $\boldsymbol{h}_L$ (the boundary between the input and output phrases). In line with the LRH, we hypothesize these features will be linear subspaces.